

Submission Worksheet

CLICK TO GRADE

<https://learn.ethereallab.app/assignment/IT114-005-F2024/it114-module-4-sockets-part-1-3/grade/bna24>

Course: IT114-005-F2024

Assignment: [IT114] Module 4 Sockets Part 1-3

Student: Bakr A. (bna24)

Submissions:

Submission Selection

1 Submission [submitted] 10/7/2024 6:05:41 PM

Instructions

^ COLLAPSE ^

Overview Video: <https://youtu.be/5a5HL0n6jek>

1. Create a new branch for this assignment
2. If you haven't, go through the socket lessons and get each part implemented (parts 1-3)
 1. You'll probably want to put them into their own separate folders/packages (i.e., Part1, Part2, Part3) These are for your reference
3. Part 3, below, is what's necessary for this HW
 3. <https://github.com/MattToegel/IT114/tree/M24-Sockets-Part3>
4. Create a new folder called Part3HW (copy of Part3)
5. Make sure you have all the necessary files from Part3 copied here and fix the package references at the top of each file
 1. Add/commit/push the branch
 2. Create a pull request to main and keep it open
6. Implement **two** of the following **server-side** activities for all connected clients (majority of the logic should be processed server-side and broadcasted/sent to all clients if/when applicable)
 1. Simple number guesser where all clients can attempt to guess while the game is active
 1. Have a /start command that activates the game allowing guesses to be interpreted
 2. Have a /stop command that deactivates the game, guesses will be treated as regular messages (i.e., guess messages are ignored)
 3. Have a /guess command that include a value that is processed to see if it matches the hidden number (i.e., /guess 5)
 1. Guess should only be considered when the game is active
 2. The response should include who guessed, what they guessed, and whether or not it was correct (i.e., Bob guessed 5 but it was not correct)
 3. No need to implement complexities like strikes

2. Coin toss command (random heads or tails)
 1. Command should be something logical like `/flip` or `/toss` or `/coin` or similar
 2. The result should mention *who* did *what* and got what *result* (i.e., Bob Flipped a coin and got heads)
3. Dice roller given a command and text format of `/roll #d#` (i.e., `/roll 2d6`)
 1. Command should be in the format of `/roll #d#` (i.e., `/roll 1d10`)
 2. The result should mention *who* did *what* and got what *result* (i.e., Bob rolled 1d10 and got 7)
4. Math game (server outputs a basic equation, first person to guess it correctly gets congratulated and a new equation is given)
 1. Have a `/start` command that activates the game allowing equation to be answered
 2. Have a `/stop` command that deactivates the game, answers will be treated as regular messages (i.e., any game related commands when stopped will be ignored)
 3. Have an answer command that include a value that is processed to see if it matches the hidden number (i.e., `/answer 15`)
 1. The response should include who answered, what they answered, and whether or not it was correct (i.e., Bob answered 5 but it was not correct)
5. Private message (a client can send a message targeting another client where only the two can see the messages)
 1. Command can be `/pm`, `/dm` followed by the user's name or an `@` preceding the user's name (clearly note which)
 2. The server should properly check the target audience and send the response to the original sender and to the receiver (no one else should get the message)
 3. Alternatively (make note if you do this and show evidence) you can add support to private message multiple people at once. Evidence should show a larger number of clients than the target list of the private message to show it works. Note to grader: if this is accomplished add 0.5 to total final grade on Canvas
6. Message shuffler (randomizes the order of the characters of the given message)
 1. Command should be `/shuffle` or `/randomize` (clearly mention what you chose) followed by the message to shuffle (i.e., `/shuffle hello everybody`)
 2. The message should be sent to all clients showing it's from the user but randomized
 1. Example: Bob types `/command` hello and everyone receives Bob: lleho
7. Fill in the below deliverables
8. Save the submission and generated output PDF
9. Add the PDF to the Part3HW folder (local)
10. Add/commit/push your changes
11. Merge the pull request
12. Upload the same PDF to Canvas

Group

100%

Group: Baseline

Tasks: 1

Points: 2

^ COLLAPSE ^

Task

100%

Group: Baseline

Task #1: Demonstrate Baseline Code Working

Weight: ~100%

Points: ~2.00

^ COLLAPSE ^

Details:

This can be a single screenshot if everything fits, or can be multiple screenshots



Columns: 1

Sub-Task

100%

Group: Baseline

Task #1: Demonstrate Baseline Code Working

Sub Task #1: Show and clearly note which terminal is the Server

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
bakrag@BakrPc: MINGW64 ~/IT114/bna24-IT114-005 (M4-Java
-Sockets)
$ java M4.Part3HW.Server
Server Starting
Listening on port 3000
Waiting for next client
Client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[29]: Thread starting
Client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[30]: Thread starting
Thread[29]: Received from my client: hi
Checking command: hi
Thread[30]: Received from my client: Hello
Checking command: Hello
```

Server Baseline Code

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Baseline

Task #1: Demonstrate Baseline Code Working

Sub Task #2: Show and clearly note which terminals are the client

Task Screenshots

4

2

1

```

bakra@BakrPc MINGW64 ~/IT114/bna24-IT114-005 (M4-Java
-Sockets)$ javac M4/Part3HW/Client.java

$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
(Server)Server: *User[29] connected*
(Server)Server: *User[30] connected*
hi
(Server)User[29]: hi
(Server)User[30]: Hello

```

Client #1 Baseline

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: Baseline

100%

Task #1: Demonstrate Baseline Code Working

Sub Task #3: Show all clients receiving the broadcasted/relayed messages

Task Screenshots

4

2

1

```

Left Terminal:
bakra@BakrPc MINGW64 ~/IT114/bna24-IT114-005 (M4-Java
-Sockets)$ javac M4/Part3HW/Client.java
$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
(Server)Server: *User[29] connected*
(Server)Server: *User[30] connected*
hi
(Server)User[29]: hi
(Server)User[30]: Hello

Middle Terminal:
bakra@BakrPc MINGW64 ~/IT114/bna24-IT114-005 (M4-Java
-Sockets)$ javac M4/Part3HW/Server.java
$ java M4.Part3HW.Server
Server Created
Server starting
Waiting for input
Server: *User[29] connected*
Server: *User[30] connected*
(Server)Server: *User[29] connected*
(Server)Server: *User[30] connected*
hi
(Server)User[29]: hi
(Server)User[30]: Hello

Right Terminal:
bakra@BakrPc MINGW64 ~/IT114/bna24-IT114-005 (M4-Java
-Sockets)$ javac M4/Part3HW/Server.java
$ java M4.Part3HW.Server
Server Created
Server starting
Waiting for input
Server: *User[29] connected*
Server: *User[30] connected*
(Server)Server: *User[29] connected*
(Server)Server: *User[30] connected*
hi
(Server)User[29]: hi
(Server)User[30]: Hello

```

Sending/Receiving

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: Baseline

100%

Task #1: Demonstrate Baseline Code Working

Sub Task #4: Include a screenshot showing you grabbed Parts 1-3 correctly and have them in your repository alongside Part3HW

Task Screenshots

4

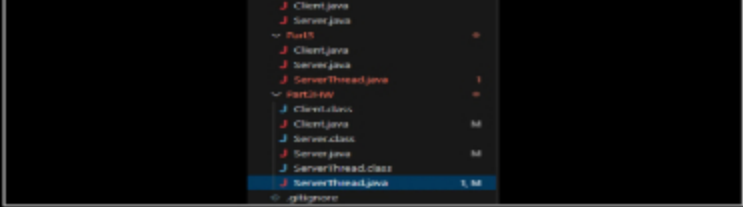
2

1

```

bakra@BakrPc MINGW64 ~/IT114/bna24-IT114-005 (M4-Java
-Sockets)$ ls -la
total 12
drwxr-xr-x 3 bakra bakra 4096 Nov 14 10:05 .
drwxr-xr-x 1 bakra bakra 4096 Nov 14 10:05 ..
drwxr-xr-x 2 bakra bakra 4096 Nov 14 10:05 Part1
-rw-r--r-- 1 bakra bakra  114 Nov 14 10:05 Part2
-rw-r--r-- 1 bakra bakra  114 Nov 14 10:05 Part3

```



Proof its in repository. It works but it shows red because of the "threadID();" method

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 1

End of Group: Baseline

Task Status: 1/1

Group

100%

Group: Feature 1

Tasks: 1

Points: 3

^ COLLAPSE ^

Task

100%

Group: Feature 1

Task #1: Solution

Weight: ~100%

Points: ~3.00

^ COLLAPSE ^

Columns: 1

Sub-Task

100%

Group: Feature 1

Task #1: Solution

Sub Task #1: Show the code related to the feature (ucid and date must be present as a comment)

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

≡ Task Response Prompt

Mention specific feature and explain sufficiently and concisely the implementation (should be aligned with code snippets)

Response:

The Coin Toss feature allows users to flip a coin by entering the /toss command. When a user types this command, the server detects it in the processCommand method, which checks for specific commands sent by clients. To generate a random outcome, I used Math.random(), producing a number between 0 and 1. If the generated number is less than 0.5, the result is "heads"; otherwise, it is "tails.". Then it sends this information to the clients using the relay method. I used a ternary operator because professor mentioned that it will be used in the midterm.

Sub-Task

100%

Group: Feature 1
Task #1: Solution
Sub Task #2: Show the feature working (i.e., all terminals and their related output)

🖼 Task Screenshots

Gallery Style: 2 Columns

4 2 1



Terminal Output For Coin Toss. Right above red lines

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 1

End of Group: Feature 1
Task Status: 1/1

Group

100%

Group: Feature 2
Tasks: 1
Points: 3

⌆ COLLAPSE ⌆

Task

100%

Group: Feature 2
Task #1: Solution
Weight: ~100%
Points: ~3.00

^ COLLAPSE ^

Columns: 1

Sub-Task

100%

Group: Feature 2
Task #1: Solution
Sub Task #1: Show the code related to the feature (ucid and date must be present as a comment)

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
126 // 10/24  
127 // 10/7/2024  
128 // Shuffle Method #1  
129 if (message.startsWith(prefix + "/shuffle ")) {  
130     String toShuffle = message.substring(beginIndex);  
131     String shuffled = shuffleMessage(toShuffle);  
132     relay("User[" + sender.getClientId() + "]: " + shuffled, sender=null);  
133     return true;  
134 } <- #129-134 if (message.startsWith("/shuffle "))
```

```
151 // 10/24  
152 // 10/7/2024  
153 // Shuffle Message #2 You: 11 minutes ago - uncommitted change  
154 private String shuffleMessage(String message) {  
155     char[] chars = message.toCharArray();  
156     StringBuilder shuffled = new StringBuilder();  
157     while (chars.length > 0) {  
158         int index = (int) (Math.random() * chars.length);  
159         shuffled.append(chars[index]);  
160         chars = removeAt(chars, index);  
161     } <- #158-162 while (chars.length > 0)  
162     return shuffled.toString();  
163 } <- #154-164 private String shuffleMessage(String message)  
164  
165 private char[] removeAt(char[] array, int index) {  
166     char[] newArray = new char[array.length - 1];  
167     for (int i = 0, j = 0; i < array.length; i++) {  
168         if (i != index) {  
169             newArray[j++] = array[i];  
170         }  
171     } <- #168-172 for (int i = 0, j = 0; i < array.length; i++)  
172     return newArray;  
173 } <- #166-174 private char[] removeAt(char[] array, int index)
```

Method being used in processCommand method

Code for the method ShuffleMessage

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Mention specific feature and explain sufficiently and concisely the implementation (should be aligned with code snippets)s

Response:

The shuffleMessage method starts by converting the input message into an array of characters and it uses StringBuilder to hold the shuffled result. There is a loop that runs if characters are left and generates a random index to select a character from the array. Then it is appended to StringBuilder while using the removeAt method to ensure it doesn't get used twice.

Sub-Task

100%

Group: Feature 2
Task #1: Solution
Sub Task #2: Show the feature working (i.e., all terminals and their related output)

Task Screenshots

Gallery Style: 2 Columns



Terminal Output For Shuffle. Below the red line.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 1

End of Group: Feature 2

Task Status: 1/1

Group

100%

Group: Misc

Tasks: 3

Points: 2

^ COLLAPSE ^

Task

100%

Group: Misc

Task #1: Reflection

Weight: ~33%

Points: ~0.67

^ COLLAPSE ^

Sub-Task

100%

Group: Misc

Task #1: Reflection

Sub Task #1: Learn anything new? Face any challenges? How did you overcome any issues?

≡ Task Response Prompt

Provide at least a few logical sentences

Response:

Learned how to utilize .getClientID and ternary operators. I struggled with getting the shuffle message to work with the StringBuilder and the removeAt method.

End of Task 1

Task



Group: Misc
Task #2: Pull request link
Weight: ~33%
Points: ~0.67

^ COLLAPSE ^

Details:

URL should end with /pull/# and be related to this assignment



Task URLs

URL #1

<https://github.com/BakrAbushaar/bna24-IT114-005/pull/8>

URL

<https://github.com/BakrAbushaar/bna24-IT114-0>

End of Task 2

Task



Group: Misc
Task #3: Waka Time (or related) Screenshot
Weight: ~33%
Points: ~0.67

^ COLLAPSE ^

Details:

Screenshot clearly shows what files/project were being worked on (the duration of time doesn't correlated with the grade for this item)



Task Screenshots

Gallery Style: 2 Columns

4

2

1



WakaTime

End of Task 3

End of Group: Misc
Task Status: 3/3

End of Assignment